

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

*I **T.Ganesh(192425246)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

T.Ganesh

Signature of the student:

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.

main.py	Visualize	Run	Output
<pre>1 n = int(input("Enter number of hosts: ")) 2 switch = input("Enter switch name: ") 3 length = float(input("Enter cable length (meters): ")) 4 5 print("\nSTAR TOPOLOGY\n") 6 7 for i in range(1, n + 1): 8 host = input(f"Enter name of Host-{i}: ") 9 print(f"{host} ----- {length} m Cat6 UTP -----> {switch}") 10 11 print("\nCentral Switch:", switch) 12 print("Number of Hosts:", n) 13 print("Cable Type: Cat6 UTP") 14 print("Cable Length:", length, "meters per host")</pre>			<pre>Enter number of hosts: 4 Enter switch name: sw1 Enter cable length (meters): 10 STAR TOPOLOGY Enter name of Host-1: pc1 pc1 ----- 10.0 m Cat6 UTP -----> sw1 Enter name of Host-2: pc2 pc2 ----- 10.0 m Cat6 UTP -----> sw1 Enter name of Host-3: pc3 pc3 ----- 10.0 m Cat6 UTP -----> sw1 Enter name of Host-4: pc4 === Session Ended. Please Run the code again ===</pre>

2. Code a function `validate\_segment(length\_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.

main.py	Visualize	Run	Output
<pre>1 def validate_segment(length_m): 2 if length_m <= 100: 3 return True 4 else: 5 print("Error: UTP segment length must be 100 meters or less.") 6 return False 7 8 print(validate_segment(50)) 9 print(validate_segment(100)) 10 print(validate_segment(120))</pre>			<pre>ERROR! True True Error: UTP segment length must be 100 meters or less. False === Code Execution Successful ===</pre>

- Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma\_log.txt`.

main.py	Visualize	Run	Output
<pre> 1 import random 2 3 hosts = ["Host-1", "Host-2", "Host-3", "Host-4"] 4 5 for attempt in range(1, 6): 6 sender1, sender2 = random.sample(hosts, 2) 7 8 print(f"\nAttempt {attempt}") 9 print(f"{sender1} and {sender2} are transmitting") 10 11 if random.choice([True, False]): 12 print("Collision detected!") 13 14 k = random.randint(0, 2) 15 backoff1 = (2 ** k) * 1 16 backoff2 = (2 ** k) * 1 17 18 print(f"{sender1} back-off interval: {backoff1} seconds") 19 print(f"{sender2} back-off interval: {backoff2} seconds") 20 else: 21 print("No collision detected.") 22 23 print("\nCSMA/CD simulation completed.") </pre>			<pre> Attempt 1 Host-4 and Host-3 are transmitting Collision detected! Host-4 back-off interval: 2 seconds Host-3 back-off interval: 2 seconds Attempt 2 Host-2 and Host-4 are transmitting Collision detected! Host-2 back-off interval: 2 seconds Host-4 back-off interval: 2 seconds Attempt 3 Host-1 and Host-4 are transmitting No collision detected. Attempt 4 Host-2 and Host-4 are transmitting Collision detected! Host-2 back-off interval: 1 seconds Host-4 back-off interval: 1 seconds </pre>

- Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.

main.py	Visualize	Run	Output
<pre> 1 import socket 2 import threading 3 4 switch_table = { 5 "Host-1": "AA:AA:AA:AA:AA:01", 6 "Host-2": "AA:AA:AA:AA:AA:02" 7 } 8 9 def server(): 10 server_socket = socket.socket(socket.AF_INET, socket. 11 SOCK_STREAM) 12 server_socket.bind(("localhost", 5000)) 13 server_socket.listen(1) 14 15 print("Switch is running...") 16 conn, addr = server_socket.accept() 17 18 while True: 19 data = conn.recv(1024).decode() 20 21 if not data: 22 break </pre>			<pre> Switch is running... Enter message from Host-1: hello host2 Frame received by switch: hello host2 Forwarding frame from Host-1 to Host-2 Switch MAC Address Table: Host-1 -> AA:AA:AA:AA:AA:01 Host-2 -> AA:AA:AA:AA:AA:02 Host-2: Message forwarded to Host-2 === Code Execution Successful === </pre>

- Create a class `StarSwitch` with methods `add\_port(host\_id)`, `remove\_port(host\_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

main.py

VisualizeRun

```

1 class StarSwitch:
2     def __init__(self):
3         self.ports = {}
4
5     def add_port(self, host_id):
6         self.ports[host_id] = True
7         print(host_id, "connected to switch")
8
9     def remove_port(self, host_id):
10         if host_id in self.ports:
11             del self.ports[host_id]
12             print(host_id, "removed from switch")
13
14     def broadcast(self, frame):
15         source = frame["source"]
16         message = frame["message"]
17
18         print("\nBroadcasting frame...")
19         for host in self.ports:
20             if host != source:
21                 print(f"Frame delivered from {source} to {host}: {message}")

```

Output

```

Host-1 connected to switch
Host-2 connected to switch
Host-3 connected to switch
Host-4 connected to switch

Enter source host: host-1
Enter message: hello

Broadcasting frame...
Frame delivered from host-1 to Host-1: hello
Frame delivered from host-1 to Host-2: hello
Frame delivered from host-1 to Host-3: hello
Frame delivered from host-1 to Host-4: hello

Enter host to remove: host-4

=== Code Execution Successful ===

```

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done